게시판 프로젝트 SSR 적용 정리

1. CSR vs SSR 개념

CSR (Client Side Rendering) - 변경 전

```
1. 브라우저가 빈 HTML을 받음
2. JavaScript 다운로드 & 실행
3. useEffect → fetch로 API 호출 (브라우저에서)
4. Loading... 표시
5. 데이터 도착 → 화면 렌더링
```

- 사용자가 **빈 화면 → Loading → 데이터**를 순서대로 봄
- SEO 불리 (검색엔진이 빈 HTML을 봄)

SSR (Server Side Rendering) - 변경 후

```
1. Next.js 서버에서 API 호출 (서버에서)
2. 데이터가 채워진 완성된 HTML을 브라우저에 전달
3. 사용자는 처음부터 데이터가 보임
```

- 사용자가 **바로 완성된 화면**을 봄
- SEO 유리 (완성된 HTML이 전달됨)

---

2. 변경된 파일 구조

Before (전부 CSR)

```
src/
  app/board/
    list/page.js          ← 'use client' (CSR)
    view/[boardId]/page.js ← 'use client' (CSR)
```

page.js 하나에 모든 코드(데이터 로딩 + UI + 인터랙션)가 들어있었음

After (SSR + 클라이언트 컴포넌트 분리)

```
src/
  app/board/
    list/page.js               ← 서버 컴포넌트 (SSR) - 초기 데이터 fetch
    view/[boardId]/page.js     ← 서버 컴포넌트 (SSR) - 게시글 데이터 fetch

  components/board/
    BoardList.js               ← 'use client' - 검색/테이블/페이징
    BoardActions.js            ← 'use client' - 목록/수정/삭제 버튼 + 삭제 모달
    Reply.js                   ← 'use client' - 댓글 목록 + 입력
    DeleteModal.js             ← 'use client' - 삭제 확인 모달
    Search.js                  ← 'use client' - 검색 영역

  components/common/
    Loading.js                 ← 로딩 UI
    Pagination.js              ← 페이지네이션 UI
```

---

3. 보기 페이지 (view) 상세 변경

Before - CSR 방식

```
'use client';  // 클라이언트 컴포넌트

import { useEffect, useState } from 'react';

export default function BoardDetailPage({ params }) {
    const [data, setData] = useState(null);     // state 필요
    const boardId = params.boardId;

    useEffect(() => {                           // useEffect로 데이터 로딩
        fetchBoardDetail(boardId);
    }, [boardId]);

    const fetchBoardDetail = async (boardId) => {
        const res = await getBoard(boardId);    // 브라우저에서 API 호출
        const result = await res.json();
        setData(result);
    };

    if (!data) return <Loading />;              // 로딩 중 표시 필요

    return (
        <div>
            {/* 본문, 버튼, 댓글, 삭제 모달 전부 여기에 */}
        </div>
    );
}
```

After - SSR 방식

```
// 'use client' 없음 → 서버 컴포넌트

import { API_BASE_URL } from "@/api/apiUrl";
import Reply from "@/components/board/Reply";
import BoardActions from "@/components/board/BoardActions";

export default async function BoardDetailPage({ params }) {  // async 함수
    const boardId = params.boardId;

    // 서버에서 직접 fetch (useEffect, useState 불필요)
    const res = await fetch(`${API_BASE_URL}/board/view/${boardId}`, {
        cache: 'no-store'   // 항상 최신 데이터
    });
    const data = await res.json();
    // Loading 불필요 - 서버에서 다 가져온 후 HTML 전달

    return (
        <div>
            {/* 본문 - 서버에서 렌더링 (인터랙션 없음) */}
            <div>{data.board.title}</div>
            <div>{data.board.content}</div>
```

```
            {/* 버튼 + 삭제 모달 - 클라이언트 컴포넌트 */}
            <BoardActions boardId={boardId} />

            {/* 댓글 - 클라이언트 컴포넌트 */}
            <Reply boardId={boardId} replyList={data.replyList} />
        </div>
    );
}
```

핵심 변경점

항목	Before (CSR)	After (SSR)
`'use client'`	있음	없음
함수 선언	`function`	`async function`
데이터 로딩	`useEffect` + `useState`	`await fetch()` 직접 호출
Loading 화면	필요	불필요
`useState`	사용	사용 불가 (서버 컴포넌트)
`onClick` 등	직접 사용	클라이언트 컴포넌트로 분리

---

4. 목록 페이지 (list) 상세 변경

After - SSR + 클라이언트 혼합

page.js (서버 컴포넌트) - 초기 1페이지 데이터만 SSR로 가져옴

```
// 'use client' 없음
import { API_BASE_URL } from "@/api/apiUrl";
import BoardList from "@/components/board/BoardList";

export default async function BoardListPage() {
    const res = await fetch(`${API_BASE_URL}/board/list?page=1`, {
        cache: 'no-store'
    });
    const data = await res.json();

    return (
        <div>
            <h1>게시글 목록</h1>
            <BoardList initialData={data} />  {/* 초기 데이터를 props로 전달 */}
        </div>
    );
}
```

BoardList.js (클라이언트 컴포넌트) - 검색/페이징 인터랙션 담당

```
'use client';

const BoardList = ({ initialData }) => {     // SSR 데이터를 초기값으로 받음
    const [data, setData] = useState(initialData);
```

```
    const [page, setPage] = useState(1);

    useEffect(() => {
        fetchBoardList(page);                // 페이지 변경 시 클라이언트에서 fetch
    }, [page]);

    return (
        <>
            <Search ... />      {/* 검색 */}
            {/* 테이블 */}
            <Pagination ... />  {/* 페이징 */}
        </>
    );
};
```

데이터 흐름

```
서버 (page.js)                    클라이언트 (BoardList.js)
   │                                      │
   ├── fetch 1페이지 데이터               │
   ├── HTML 완성                          │
   ├── initialData props 전달  ──────→ useState(initialData)
   │                                      │
   │                                      ├── 2페이지 클릭 → useEffect → fetch
   │                                      ├── 검색 클릭 → fetch
   │                                      └── setData → 리렌더링
```

---

5. 컴포넌트 분리 이유 (SSR에서 왜 분리가 필요한가)

서버 컴포넌트에서는 **useState, useEffect, onClick** 등을 사용할 수 없다. 따라서 인터랙션이 필요한 부분은 반드시 `'use client'` 클라이언트 컴포넌트로 분리해야 한다.

view 페이지 분리 구조

```
page.js (서버 컴포넌트, SSR)
│
├── 본문 영역 (카테고리, 제목, 작성자, 내용, 첨부파일)
│   → 서버에서 렌더링 (인터랙션 없음, 데이터 표시만)
│
├── BoardActions (클라이언트 컴포넌트)
│   → onClick 필요 (목록/수정/삭제 버튼)
│   → useState 필요 (삭제 모달 열기/닫기)
│   └── DeleteModal (클라이언트 컴포넌트)
│       → useState 필요 (비밀번호 입력)
│       → API 호출 (삭제)
│
└── Reply (클라이언트 컴포넌트)
    → useState 필요 (댓글 입력, 댓글 목록)
    → API 호출 (댓글 등록, 목록 새로고침)
```

---

6. Reply 컴포넌트 변경 (콜백 → 자체 관리)

Before (CSR) - 부모가 데이터 관리

```
page.js: fetchBoardDetail() → setData() → data.replyList를 Reply에 전달
Reply: 댓글 등록 성공 → onGetReplies() 콜백으로 부모에게 새로고침 요청
```

After (SSR) - Reply가 자체 관리

```
page.js (서버): data.replyList를 Reply에 초기값으로 전달
Reply: replyList를 자체 state로 관리
Reply: 댓글 등록 성공 → 직접 API 호출 → setReplyList()로 자체 업데이트
```

서버 컴포넌트의 함수를 클라이언트 컴포넌트에 콜백으로 넘길 수 없기 때문에, Reply가 스스로 댓글 목록을 다시 가져오도록 변경함.

```
// Reply.js
const Reply = ({ boardId, replyList: initialReplyList }) => {
    const [replyList, setReplyList] = useState(initialReplyList);

    const handleReplyRegister = async () => {
        const res = await createReply(boardId, replyContent);
        if (res.ok) {
            // 부모 콜백 대신 직접 최신 데이터 조회
            const boardRes = await getBoard(boardId);
            const data = await boardRes.json();
            setReplyList(data.replyList);
        }
    };
};
```

---

7. 페이지 이동 시 주의점 (router.push vs window.location.href)

문제

SSR 페이지로 `router.push()` 를 사용하면 Next.js의 **라우터 캐시** 때문에 이전에 SSR로 받은 데이터를 그대로 보여줌 → 등록/수정/삭제 후 최신 데이터가 안 보임

해결

데이터 변경 후에는 `window.location.href` 로 하드 네비게이션

```
// 데이터 변경 없이 단순 이동 → router.push (빠름)
router.push("/board/list");
router.push(`/board/view/${boardId}`);

// 등록/수정/삭제 후 이동 → window.location.href (최신 데이터 보장)
window.location.href = "/board/list";
window.location.href = `/board/view/${boardId}`;
```

적용 위치

파일	상황	방식
write/page.js	등록 성공 후 목록으로	`window.location.href`
modify/page.js	수정 성공 후 보기로	`window.location.href`
BoardActions.js (DeleteModal)	삭제 성공 후 목록으로	`window.location.href`

BoardActions.js	목록/수정 버튼 클릭	`router.push` (단순 이동)

---

8. cache: 'no-store' 옵션

```
const res = await fetch(url, {
    cache: 'no-store'  // 매 요청마다 서버에서 새로 fetch
});
```

- Next.js는 기본적으로 fetch 결과를 캐싱함
- `cache: 'no-store'` 를 넣으면 매번 최신 데이터를 가져옴
- 게시판처럼 데이터가 자주 변경되는 페이지에 필수

---

9. 전체 요약

서버 컴포넌트 (SSR)

- `'use client'` 선언 없음 (기본값이 서버 컴포넌트)
- `async function` 으로 선언
- `await fetch()` 로 서버에서 직접 데이터 로딩
- `useState` , `useEffect` , `onClick` 사용 불가
- Loading 상태 불필요

클라이언트 컴포넌트

- `'use client'` 선언 필수
- `useState` , `useEffect` , `onClick` 등 인터랙션 가능
- 서버 컴포넌트에서 props로 초기 데이터를 받을 수 있음

분리 기준

- **인터랙션 없음** (데이터 표시만) → 서버 컴포넌트에 유지
- **인터랙션 있음** (클릭, 입력, 상태 변경) → 클라이언트 컴포넌트로 분리